Parallel & Distributed Computing Lab

IMSciences

Lab Instructor: Saim Sanan

Week 4, Lab 1

OpenMP: Barrier, Atomic, and Locks

---

1 Ideas

- **barrier** — every thread stops at that line until *all* threads have arrived, then all continue. An `after` line comes only after every `before` line.
- **atomic** — one simple update is exclusive (example: `sum += i`). Same idea as `critical`, but for one statement, usually cheaper. Not a whole block of code.
- **omp_lock_t** — a lock you create: `init`, `set` (take), `unset` (give back), `destroy`. `critical` was a lock with no name.
- **reduction** is still the right tool for a fat sum. Atomic / critical / lock protect a *shared* update.

barrier = all threads wait, then all continue. atomic = one add at a time. lock = take the lock, update, then release it.

2 Commands

Terminal

```
nano omp_barrier.c
gcc -fopenmp omp_barrier.c -o barrier
./barrier

nano omp_atomic.c
gcc -fopenmp omp_atomic.c -o atomic
./atomic

nano omp_lock.c
gcc -fopenmp omp_lock.c -o lock
./lock
```

nano: Ctrl+O, Enter (save), Ctrl+X (quit).
Forgot `./` → command not found.
Forgot `-fopenmp` → error on `omp.h`, or still one thread.

3 Barrier

Four `before` lines, then four `after` lines. Thread ids can shuffle inside each group.
C

```
#include <stdio.h>
#include <omp.h>

int main(void) {
    omp_set_num_threads(4);

    #pragma omp parallel
```

```
    {
        printf("before barrier thread %d\n", omp_get_thread_num());

        #pragma omp barrier

        printf("after barrier thread %d\n", omp_get_thread_num());
    }

    return 0;
}
```

If an `after` prints before all four `before` lines, the barrier is missing.

4 Atomic

Same expected total as the critical program, $N = 100000$:

$$\frac{N(N-1)}{2} = 4999950000$$

C

```
#include <stdio.h>
#include <omp.h>

#define N 100000

int main(void) {
    long i;
    long sum = 0;

    omp_set_num_threads(4);

    #pragma omp parallel for
    for (i = 0; i < N; i++) {
        #pragma omp atomic
        sum += i;
    }

    printf("sum = %ld\n", sum);
    printf("expected = %ld\n", (long)N * (N - 1) / 2);
    return 0;
}
```

Run twice. Printed `sum` must equal expected both times.

5 Lock

Loop `i = 0` to 4. Expected $0 + 1 + 2 + 3 + 4 = 10$.

C

```
#include <stdio.h>
#include <omp.h>

int main(void) {
    int i;
    int shared_var = 0;
    omp_lock_t lock;

    omp_set_num_threads(4);
    omp_init_lock(&lock);
```

```
    #pragma omp parallel for
    for (i = 0; i < 5; i++) {
        omp_set_lock(&lock);
        shared_var += i;
        omp_unset_lock(&lock);
    }

    omp_destroy_lock(&lock);

    printf("shared_var = %d\n", shared_var);
    printf("expected = 10\n");
    return 0;
}
```

Run twice. `shared_var` must be 10. If the program hangs, `unset` is missing.